

Deep Learning for Solving Dynamic Models

Session 6: Deep surrogates and Gaussian processes

Simon Scheidegger

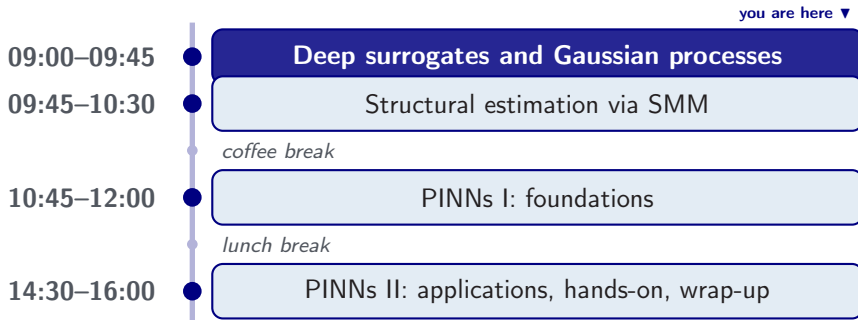
HEC, University of Lausanne

Based on Chen, Didisheim & Scheidegger (2026), *Journal of Financial Economics* 177, 104222

University of Padova

September 24, 2026

Day 2 at a Glance



Roadmap of This Lecture (45 min)

I. What is a surrogate? (10 min)

- ▶ The bottleneck, the look-up table, the four steps
- ▶ One built on the Brock–Mirman model (06_01)
- ▶ Two kinds: labelled, and trained on the equations

II. GP regression (23 min)

- ▶ Conditioning a Gaussian is regression
- ▶ Kernels, the posterior, hyperparameters
- ▶ Fifteen lines of code (06_02)

Notebooks in `day2/code/06_surrogates_and_gps/`: `06_01_Surrogates_The_Idea.ipynb`,
`06_02_GP_Regression.ipynb`

III. GP or network? (9 min)

- ▶ The same task, both surrogates, measured
- ▶ The decision rule: what a label costs
- ▶ Active learning, in one frame

Finger exercise and hands-on (3 min)

Learning Objectives

By the end of this lecture you should be able to:

- ▶ Explain a surrogate as a cheap, differentiable stand-in for an expensive model, and say which loop it takes the model out of
- ▶ Build one in four steps, and treat model parameters as extra inputs so that one fit serves a whole family of questions
- ▶ Write down Gaussian-process regression as conditioning a Gaussian, choose a kernel, and read a credible band
- ▶ Decide between a Gaussian process and a network from the cost of a label, the dimension, and whether an error bar is needed

Part I

What Is a Surrogate?

The Bottleneck Is Always the Same

Expensive to evaluate

One evaluation of a structural model means solving a Bellman equation, a system of Euler equations, a PDE, or a long simulation: seconds for the Brock–Mirman model of Session 2, hours for a serious one.

And we almost never want just one evaluation: **estimation** sweeps hundreds or thousands of candidate parameter vectors (Session 7); **sensitivity analysis** propagates a distribution over parameters; **policy design** optimizes over a space of policy rules.

The one advantage we have

We own the model. We can generate as much training data as we are willing to pay for, and the labels are exact up to the solver's tolerance. Chen, Didisheim & Scheidegger (2026) re-estimate an option-pricing model every day for a year this way; at the model's own speed that was not feasible.

Look-up Tables $\rightarrow$ Surrogates

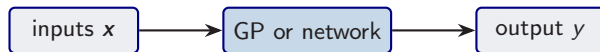
The old idea: compute once, then interpolate.

x	$\Phi(x)$
-2.0	0.0228
-1.0	0.1587
0.0	0.5000
1.0	0.8413
2.0	0.9772

The normal CDF table in the back of every old statistics textbook. Nobody re-derives Φ ; they look it up.

A surrogate is a look-up table with a fitted function in place of the table:

$$\phi(\mathbf{x}) \approx y = f(\mathbf{x}), \quad \mathbf{x} \in \text{a box.}$$



A table has one entry per grid point and dies of the curse of dimensionality. A fitted function is **continuous**, **differentiable**, and its cost grows with the complexity of f , not with n_g^d . Which function to fit is Part III.

Building One, in Four Steps

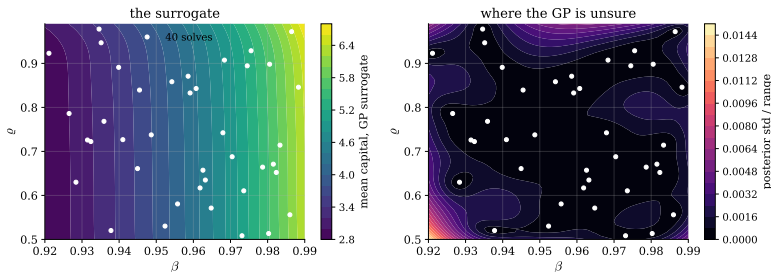
1. **Design.** Choose inputs $\mathbf{x}_i$, $i = 1, \dots, N$, spread over the box: uniform, Latin hypercube, Sobol' sequence, or (Part III) one at a time where the surrogate is least sure.
2. **Label.** Evaluate the expensive model: $y_i = f(\mathbf{x}_i)$. **This is the step that costs money**, and it happens once.
3. **Fit.** A Gaussian process (closed form) or a network (minimize $\frac{1}{N} \sum_i |\phi(\mathbf{x}_i) - y_i|^2$).
4. **Validate.** Fresh labels the fit never saw: mean *and* maximum error over the box, not a training loss.

Parameters as inputs

Do not build one surrogate per parameter vector. Put the parameters θ into $\mathbf{x}$ alongside the states, so that one fit answers for every θ in the box. Session 7 estimates θ by moving along that axis of one surrogate; Chen, Didisheim & Scheidegger (2026) call them *pseudo-states*.

One Built on the Brock–Mirman Model

The expensive model: the stochastic Brock–Mirman economy of Session 2 at a given (β, ϱ) , solved by a reference method (endogenous grid, cubic splines, Euler errors below 10^{-6}), about 1.7 s per solve. **Quantity of interest:** the mean capital stock and the mean investment share along a 2,000-period simulation, $q(\beta, \varrho)$.



Notebook 06_01. Left: the GP surrogate's mean capital from **40 solves** (white dots); a 30×30 grid would have cost 25 minutes. Right: its posterior standard deviation, largest away from the design. On 60 fresh solves the GP's mean error is $1.1 \cdot 10^{-3}$ of the quantity's range, the network's $3.9 \cdot 10^{-3}$, both fit to the same 40 labels.

Two Kinds of Surrogate

Labelled: a regression of f

- ▶ Inputs and outputs from the expensive solver
- ▶ Any approximator: GP, network, polynomial
- ▶ Summarizes what the model implies (q , moments, welfare)
- ▶ Cost = number of solves

This session: 06_01, 06_02.

Trained on the equations: the solution itself

- ▶ A policy network (state, θ) $\mapsto$ decision
- ▶ Loss = the Euler residual at sampled states, as in the DEQNs of Day 1
- ▶ No solver, no labels; θ is one more input
- ▶ Cost = training once

Session 7: 07_01, 07_02; then compared with GPs of the moment map.

Same validation for both

Compare with a reference solution at several θ , on the states the model actually visits. The Euler residual is a training loss, not an accuracy measure.

Part II

Gaussian-Process Regression

Conditioning Is Regression

Joint:

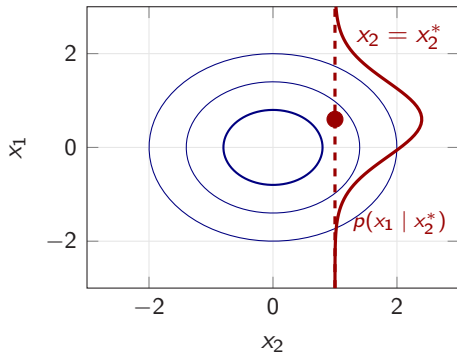
$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}, \begin{pmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{pmatrix}\right)$$

Conditional:

$$\mu_{1|2} = \mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(x_2 - \mu_2)$$

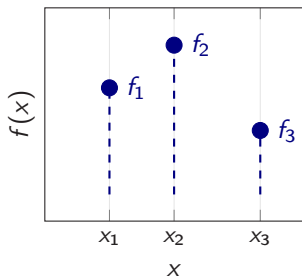
$$\Sigma_{1|2} = \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21}$$

These two lines are the entire engine of GP regression. Everything that follows is bookkeeping about how to build Σ .



Slicing the joint at $x_2 = x_2^*$ leaves a 1D Gaussian: a prediction *and* an error bar.

From Observations to Interpolation



Assume the function values are jointly Gaussian:

$$\begin{pmatrix} f_1 \\ f_2 \\ f_3 \end{pmatrix} \sim \mathcal{N}\left(0, \begin{pmatrix} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \\ K_{31} & K_{32} & K_{33} \end{pmatrix}\right)$$

Nearby points should be more correlated than distant ones. With a squared-exponential kernel and $\ell = 2$:

$$K = \begin{pmatrix} 1.00 & 0.76 & 0.22 \\ 0.76 & 1.00 & 0.61 \\ 0.22 & 0.61 & 1.00 \end{pmatrix}$$

Build the covariance from a **kernel**, $K_{ij} = k(x_i, x_j)$. Conditioning on the observed f_i then predicts everywhere else.

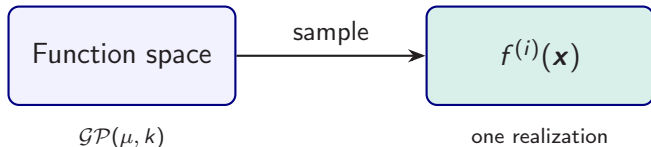
A Gaussian Process: A Distribution over Functions

Definition

A **Gaussian process** is a collection of random variables, any finite subset of which is jointly Gaussian:

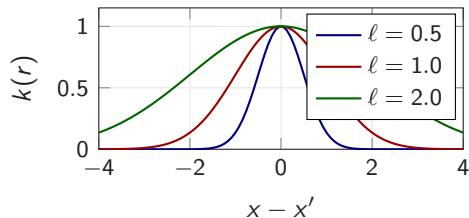
$$f(\mathbf{x}) \sim \mathcal{GP}(\mu(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')).$$

- ▶ **Mean function** $\mu(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})]$, almost always 0 after centering the data.
- ▶ **Covariance function** $k(\mathbf{x}, \mathbf{x}') = \mathbb{E}[(f(\mathbf{x}) - \mu(\mathbf{x}))(f(\mathbf{x}') - \mu(\mathbf{x}'))]$, where all the modeling lives.



Kernels: Where the Prior Lives

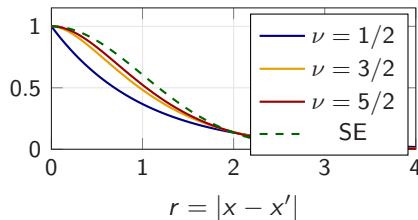
Squared exponential



$k_{\text{SE}}(r) = \sigma_f^2 \exp(-r^2/2\ell^2)$. Larger $\ell \Rightarrow$ smoother; samples are **infinitely differentiable**, often too optimistic.

Choosing the kernel is not a technicality: it *is* the model. Assume more smoothness than the truth has, and the GP interpolates confidently straight through a kink.

Matérn family, $\ell = 1$



ν sets differentiability: samples are $\lceil \nu \rceil - 1$ times differentiable. $\nu = 5/2$ or $3/2$ is the sane default for value and policy functions.

GP Regression: The Posterior in Two Lines

Joint distribution of training values $\mathbf{f}$ and test values $\mathbf{f}_*$:

$$\begin{pmatrix} \mathbf{f} \\ \mathbf{f}_* \end{pmatrix} \sim \mathcal{N}\left(0, \begin{pmatrix} K & K_* \\ K_*^\top & K_{**} \end{pmatrix}\right), \quad K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j).$$

Condition on what you observed: the two lines from three slides ago:

$$\mu_{*|\mathcal{D}} = K_*^\top K^{-1} \mathbf{f}$$

$$\Sigma_{*|\mathcal{D}} = K_{**} - K_*^\top K^{-1} K_*$$

Read the variance line carefully

- ▶ Near the data: $K_*^\top K^{-1} K_* \approx K_{**}$, so the variance **collapses**.
- ▶ Far from the data: $K_*^\top K^{-1} K_* \approx 0$, so the variance **reverts to the prior**.

Note what this means: the GP *knows it is extrapolating*, and says so, without ever seeing the true function out there. No trained network does this by default.

Noise, the Nugget, and Numerical Stability

Observation model. If labels carry noise, $y = f(\mathbf{x}) + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma_y^2)$, then $\text{cov}[y_p, y_q] = k(\mathbf{x}_p, \mathbf{x}_q) + \sigma_y^2 \delta_{pq}$, so everywhere replace K by $K_y = K + \sigma_y^2 I$:

$$\bar{f}_* = \mathbf{k}_*^\top (K + \sigma_y^2 I)^{-1} \mathbf{y}, \quad \sigma_*^2 = k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}_*^\top (K + \sigma_y^2 I)^{-1} \mathbf{k}_*.$$

Noiseless ($\sigma_y^2 = 0$)

- ▶ Interpolates the training points exactly
- ▶ Variance is zero at training inputs
- ▶ Needs K strictly positive definite

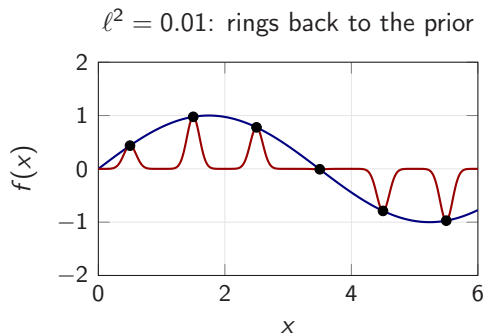
Noisy ($\sigma_y^2 > 0$)

- ▶ Smooths through the training points
- ▶ Variance positive everywhere
- ▶ Much better conditioned

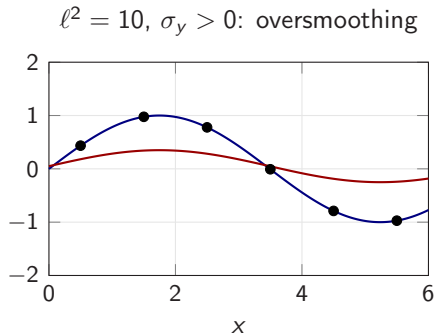
In practice

Even for a “noiseless” surrogate, add a jitter of $\sigma_y^2 \sim 10^{-6}$. **And be honest about whether it really is noiseless:** if the labels came from a Monte Carlo simulation or an iterative solver stopped at a tolerance, they carry noise, and pretending otherwise makes the Cholesky factorization fail before it makes the statistics wrong.

Getting the Length Scale Wrong



Too short. The GP interpolates every point exactly and then **falls back to the prior mean in between**. Perfect training fit, useless predictions, and the credible band (previous slide) balloons in exactly those gaps.



Too long. The posterior mean flattens and no longer passes through the data. The band is narrow and **confidently wrong**, which is the more dangerous failure of the two.

Learning the Hyperparameters, and a Free Diagnostic

Marginal likelihood: how ℓ , σ_f , σ_y get chosen

$$\log p(\mathbf{y} \mid X) = \underbrace{-\frac{1}{2}\mathbf{y}^\top K_y^{-1}\mathbf{y}}_{\text{data fit}} \underbrace{-\frac{1}{2}\log |K_y|}_{\text{complexity penalty}} \underbrace{-\frac{N}{2}\log(2\pi)}_{\text{constant}}$$

The two terms pull against each other: **an automatic Occam's razor**, with closed-form gradients and no validation split spent. Optimize with restarts; it is multimodal.

Leave-one-out, for free

$$\mu_{-i}(\mathbf{x}_i) - y_i = -\frac{[K_y^{-1}\mathbf{y}]_i}{[K_y^{-1}]_{ii}}, \quad \sigma_{-i}^2(\mathbf{x}_i) = \frac{1}{[K_y^{-1}]_{ii}}.$$

These need $\text{diag}(K_y^{-1})$, so *all* N leave-one-out residuals cost one further $\mathcal{O}(N^3)$ solve, the same order you already paid, and **no refitting and no held-out data**. Doing N honest refits would be $\mathcal{O}(N^4)$. **This is the number to report for a surrogate.**

Closed form: Rasmussen & Williams (2006), §5.4.2, in ../readings.

GP Regression in Fifteen Lines

```
import numpy as np
from scipy.linalg import cho_factor, cho_solve

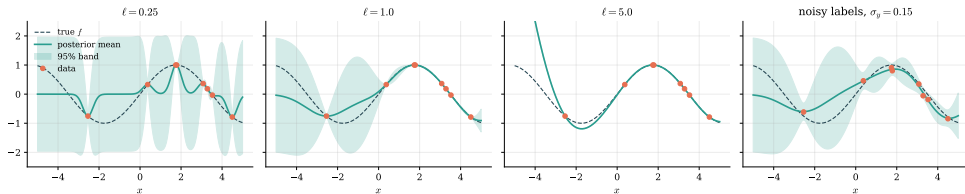
def kernel(X1, X2, l=1.0, sf=1.0):
    sq = np.sum(X1**2, 1).reshape(-1, 1) \
        + np.sum(X2**2, 1) - 2 * X1 @ X2.T
    return sf**2 * np.exp(-0.5 * sq / l**2)

K = kernel(X_tr, X_tr) + 1e-8 * np.eye(N)      # jitter
L, low = cho_factor(K, lower=True)             #  $O(N^3)$ , once
alpha = cho_solve((L, low), y_tr)              # the weights

Ks = kernel(X_tr, X_te)
mu = Ks.T @ alpha                             # posterior mean
v = cho_solve((L, low), Ks)
var = kernel(X_te, X_te) - Ks.T @ v           # posterior variance
```

No training loop, no learning rate, no architecture: the whole model is the kernel and one Cholesky factorization. Compare Algorithm 2.1 in Rasmussen & Williams (2006); notebook 06_02 builds this and checks it against scikit-learn.

Prediction and Credible Bands

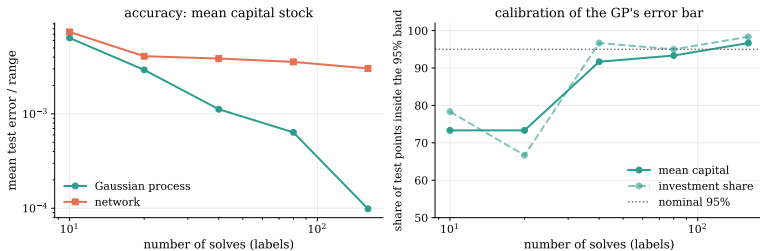


Notebook 06_02, eight exact observations of $\sin(0.9x)$, squared-exponential kernel. The band pinches shut at each observation and opens between them. Left to right: length scale too short (returns to the prior mean between points), right, too long (confidently wrong), and noisy labels with $\sigma_y = 0.15$ (smooths through the points).

Part III

GP or Network?

Few Labels: the GP Is Better, and Calibrated



1. **Accuracy.** Same labels, same 60 test solves. At 40 labels the GP's mean error is $1.1 \cdot 10^{-3}$ of the range, the network's $3.9 \cdot 10^{-3}$; at 160 the GP is at $1.0 \cdot 10^{-4}$, the network still at $3.0 \cdot 10^{-3}$.
2. **Calibration.** From 40 labels on, the GP's 95% band contains the truth at 92 to 98% of the test points: the error bar can be trusted, and it says where the surrogate is unsure.
3. **Why.** The GP passes through the labels exactly under a smoothness prior; with few labels that prior beats the network's flexibility, which is limited here by its training, not its size. With thousands of labels and many inputs the balance flips (Part I, Session 7).

Notebook 06_01. GP: Matérn 5/2, one length scale per input, marginal likelihood. Network: 2×32 tanh, Adam, 3,000 epochs.

The Decision Rule Is the Cost of a Label

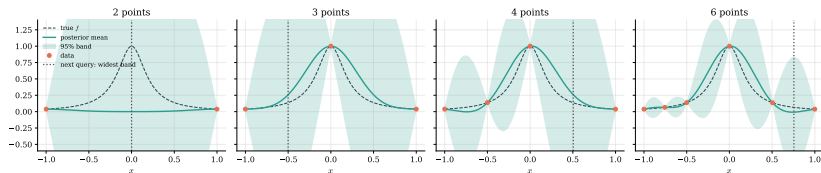
Criterion	Gaussian process	Network
Uncertainty	✓posterior variance, calibrated	✗needs ensembles
Hyperparameters	✓marginal likelihood, closed-form gradient	●search (Session 3)
Fitting	✓one Cholesky factorization	●SGD, epochs, seeds
Scaling in N	✗ $\mathcal{O}(N^3)$, practical limit $\sim 10^4$	✓ $\mathcal{O}(N)$ per epoch
Scaling in d	✗degrades past $d \approx 10$	✓ $d \gg 100$
Few labels ($N < 100$)	✓	✗

In order

1. **Price one label.** Seconds, or hours? Hours means N will be small, and the $\mathcal{O}(N^3)$ never bites.
2. **Count the inputs** after adding the parameters. Ten or fewer favours the GP.
3. **Ask whether a decision rides on the surrogate.** If so, you need the error bar.
4. **Labels cheap and d large:** network. **Labels expensive and d small:** GP. **Both:** a policy network solves the model, a GP maps its output to the quantity of interest.

Active Learning: Let the Band Choose the Next Label

With the kernel fixed, the posterior variance depends on the design alone, so it can be consulted *before* the next label is paid for: **query where the band is widest, refit, repeat.**



Notebook 06_02: $f(x) = 1/(1 + 25x^2)$ on $[-1, 1]$, squared-exponential kernel with $\ell = 0.3$ fixed, starting from the two endpoints; the dotted line is the next query.

- **What it does.** After the endpoints it takes the centre, then the quarter points: the widest band is always the largest gap in the design. After 8 points: $\{-1, -0.76, -0.5, 0, 0.23, 0.51, 0.76, 1\}$.
- **What it does not do on an interval.** A uniform design with 8 points is already that, and does slightly better (mean absolute error 0.043 against 0.072). The loop pays where no uniform design exists: irregular domains, unequal length scales, many inputs (Scheidegger & Bilonis, 2019).

Finger Exercise: How Many Labels?

PAUSE: 5 min

The setting

You must map a welfare objective over a **6-dimensional** policy box. One evaluation means simulating a stochastic OLG model, ≈ 4 hours of wall-clock time. You have a 64-core machine and **two weeks**.

1. Roughly how many evaluations can you afford?
2. How many points per dimension would a tensor grid give you at that budget?
3. Which approximator does that budget select, and why?

Solution: How Many Labels?

1. The budget

14 days $\times$ 24 h = 336 h of wall-clock time; at 64 cores that is $\sim 21,500$ core-hours, so about **5,000 evaluations** at 4 h each. Call it a few thousand, allowing for failures and re-runs.

2. What a grid does with them

$5,376^{1/6} \approx 4.2$ points per dimension. Four per axis will not resolve a curved welfare surface, and refining to a still-inadequate six costs $6^6 = 46,656$ evaluations = 187,000 core-hours, or **four months** on the same 64 cores.

3. Which approximator

A few thousand points is far too few to train a network well, and comfortably inside the $\mathcal{O}(N^3)$ range of an exact GP. **Choose the GP**, and since every point is precious, do not place them on a grid at all: let the posterior variance choose them.

Hands-On: What to Run

Notebooks in `day2/code/06_surrogates_and_gps`, CPU-only. Both ship with the teaching preset stored; in class set `RUN_MODE = "smoke"` first.

- ▶ **Surrogates, the idea** (6 min). Time one solve of the Brock–Mirman model; label a Latin-hypercube design; fit a GP and a network to the same labels; watch both errors fall with the number of solves, and check whether the GP's band is calibrated. 06_01
- ▶ **GP regression** (28 s). The kernel and prior samples; the posterior in two lines, checked against scikit-learn; length scale and noise; Matérn against squared-exponential on a kink; two inputs with one length scale each; the active-learning loop. 06_02

Next session

The surrogate becomes the model's own policy, with ϱ and then (β, ϱ) as inputs, and is simulated inside an SMM estimator.



Questions?

Simon Scheidegger

HEC, University of Lausanne

simon.scheidegger@unil.ch · sischei.github.io

Next up, Session 7: *Structural estimation via the simulated method of moments.*

Materials:

github.com/sischei/deep_learning_padova_0926